School of Computer Science and Engineering

Faculty of Engineering

The University of New South Wales

Modelling the impact of resources on Docker Containers

by

Sadat Muhtasim Kabir

Thesis submitted as a requirement for the degree of

Bachelor of Engineering (Honours) in Computer Engineering

Submitted: November 2025

Supervisor: A/Prof. Chun-Tung Chou Student ID: z5443562

Abstract

Capacity planning determines how computing systems must be provisioned to maintain acceptable performance under load. Although courses such as COMP9334 teach queueing theory, operational laws, and discrete-event simulation (DES), students rarely see how these models relate to the behaviour of real systems. This limits their ability to understand modern challenges such as bursty workloads, saturation effects, and tail-latency escalation that dominate performance in today's distributed environments.

This thesis establishes the foundation for bridging that gap through a lightweight, empirical experimentation framework that runs on student-level hardware. The platform enables controlled performance measurement under varying workloads, providing real traces of latency, utilisation, and service-time behaviour. These measurements support a hybrid modelling approach in which machine-learning techniques learn empirical performance characteristics, while DES incorporates those characteristics to predict system behaviour.

The work in Thesis A develops the motivation, literature context, and initial experiments required to support this framework. Beyond its educational value, the project also targets a contemporary industry problem: predicting latency, saturation thresholds, and tail-behaviour in modern systems using data-driven models. The resulting methodology aims to unify theory and practice, offering students—and practitioners—a modern and realistic approach to capacity planning.

Acknowledgements

This research was carried out with guidance and support from many people. I thank my supervisor Chun Tung Chou and my assessor Mahbub Hassan for advice and feedback during the design and experimental phases of the project. I am grateful to the COMP9334 teaching team for discussions that motivated this work and to peers who helped test the Docker experiments and reproduce results on diverse platforms.

Open-source projects and communities provided indispensable tools and examples: cAdvisor, Prometheus, Grafana, Vegeta and Docker made the experiments possible and their developers deserve acknowledgement.

Finally, I thank friends and family for their patience during the writing of this thesis.

Abbreviations

BE Bachelor of Engineering

CPU Central Processing Unit

DES Discrete-Event Simulation

M/G/1 Queue model with Markovian arrivals, General service-time, 1 server

p50,p90,p95,p99 Latency percentiles (median, 90th, 95th, 99th)

cAdvisor Container Advisor (live container metrics collector)

Prometheus Time-series monitoring system

Grafana Metrics visualization dashboard

ML Machine Learning

WSL2 Windows Subsystem for Linux 2

HTTP HyperText Transfer Protocol

OCI Open Container Initiative

SLO Service-Level Objective

SRE Site Reliability Engineering

DAG Directed Acyclic Graph

VM Virtual Machine

vCPU Virtual CPU

HPC High-Performance Computing

MLASP Machine Learning-Assisted Capacity Planning

TSDB Time-Series Database

KDE Kernel Density Estimation

GMM Gaussian Mixture Model

ARIMA AutoRegressive Integrated Moving Average

LSTM Long Short-Term Memory neural network

API Application Programming Interface

URL Uniform Resource Locator

M/M/1 Queue with Markovian arrivals, Markovian service-time, 1 server

M/M/c Queue with Markovian arrivals, Markovian service-time, c servers

G/G/1 Queue with General arrivals, General service-time, 1 server

Contents

1 Introduction **1**

2 Background **3**

2.1 Capacity Planning Concepts 3

2.2 Queueing Theory Foundations 4

2.3 Operational Laws 4

2.4 Discrete-Event Simulation 5

2.5 Previous Work 6

3 Literature Review **7**

3.1 Introduction 7

3.2 Capacity Planning in Modern Industry 8

3.3 Educational Approaches and Their Limitations 9

3.4 Ideal vs. Practical Experimental Environments 10

3.5 Why Lightweight Containers Are the Suitable Choice 11

3.6 Measurement Tools: Choosing cAdvisor, Prometheus, and Grafana . . . 13

3.7 Hybrid ML–DES Models in Prior Research 15

3.8 Synthesis: Gap in the Literature 16

4 Preliminary Preparation **19**

4.1 Experimental Environment 20

4.1.1 Microservice and Container Configuration 20

4.1.2 Load Generation with Vegeta 20

4.1.3 Observability Stack 21

4.2 Load-Test Results 22

4.2.1 Latency Summary 22

4.2.2 Latency Curves 23

4.3 Queueing-Theoretic Interpretation 24

4.4 Implications for the Thesis Aims 25

4.5 Threats to validity 27

5 Plans for Thesis B and C **28**

5.1 Work Breakdown 28

5.2 Eighteen–Week Gantt Chart 30

6 Conclusion **31**

Bibliography **33**

List of Figures

4.1 Mean, p90, and p99 latency versus request rate for the Dockerised microservice. 24

5.1 Planned timeline for Thesis B and Thesis C (18 weeks). 30

List of Tables

3.1 Limitations of alternative technologies for student-friendly, reproducible performance-modelling environments. 13

3.2 Alignment of the observability stack with thesis measurement goals. . . 14

4.1 Latency measurements for the Dockerised microservice under different request rates (Vegeta, 60 s runs). 22

4.2 Mean, p99, and maximum latency as a function of request rate. 22

Chapter 1

Introduction

Capacity planning is concerned with predicting how system performance changes as workloads grow and resources become constrained. In modern microservice-based systems, this task is increasingly challenging: service times shift under CPU pressure, tail latencies become heavy-tailed, and saturation points move as demand fluctuates. Understanding these effects is essential for anticipating overload, ensuring responsiveness, and making informed resource-allocation decisions.

Although courses such as COMP9334 introduce the foundations of queueing theory, operational laws, and discrete-event simulation (DES), students rarely encounter a setting where these behaviours can be observed directly. Without empirical measurements, key ideas—such as utilisation growth, queue formation, tail-latency amplification, and the onset of saturation—remain abstract. This disconnect makes it difficult to appreciate how analytical models behave when applied to real systems exhibiting variability, contention, and non-linear delay characteristics.

This thesis addresses that gap by constructing a controlled experimental platform around a simple microservice exposed to varying workloads and CPU constraints. The resulting performance traces—service times, latency distributions, utilization, and indicators of queue buildup—form the empirical basis for the modeling work. Using these traces, the thesis develops a hybrid modeling approach that combines machine-learning

methods with DES: ML learns how service-time distributions evolve under resource pressure, while DES uses these learned distributions to simulate queueing dynamics, tail behaviour, and saturation thresholds. This combination captures phenomena that neither classical queueing models nor purely data-driven methods can fully explain in isolation.

By comparing empirical measurements with analytical predictions, DES outputs, and learned performance models, the study shows how classical theory and modern data-driven techniques complement each other in contemporary capacity planning. The unified case study provides students with a realistic environment in which they can observe performance behaviour first-hand, validate theoretical concepts, and explore predictive approaches to scaling, resource allocation, and overload prevention. We also address the fact that existing work on container benchmarking provides measurements but not predictive modeling; DES research provides modeling frameworks but lacks integration with real containerized systems; and ML-assisted DES exists but not in the context of computing performance or microservices. This thesis develops an empirical–ML–DES hybrid capable of forecasting performance degradation and tail behaviour under changing workload and resource conditions—a capability neither traditional queueing models nor current ML systems provide alone. Although designed to run on student machines, the framework addresses a genuinely open industrial problem: predicting how microservice performance evolves under CPU contention and fluctuating demand.

The remainder of this thesis unfolds in a structured progression. First, the background chapter introduces the foundational concepts needed to interpret modern capacity-planning behaviour—covering utilisation laws, queueing models, operational analysis, and simulation principles. Next, a focused literature review surveys industrial practises and prior academic work, highlighting how contemporary systems are monitored, modelled, and scaled, and identifying the gaps that motivate the framework developed here. This is followed by the detailed project plan and methodology, which outline how empirical traces were collected, how modeling components were designed, and how the hybrid DES–ML approach will be implemented.

Chapter 2

Background

For the purpose of this thesis, it was necessary for us to go through some of the content from the course COMP9334, which I have compiled below.

2.1 Capacity Planning Concepts

Capacity planning concerns predicting how a system will perform under different workloads and determining the resources needed to meet service objectives. Key terminology includes:

- **Throughput** (X): the rate at which a system completes work.
- **Service demand** (D): the total service time a job requires from a resource.
- **Utilisation** (U): the fraction of time a resource is busy.
- **Bottleneck device**: the resource with the largest service demand, which limits maximum throughput.
- **Saturation**: the point at which utilisation approaches 100%, causing rapid growth in response time.

These concepts provide the foundation for evaluating system behaviour before, during, and after load increases.

2.2 Queueing Theory Foundations

Queueing theory models how jobs compete for shared resources. A queue is typically denoted using *Kendall notation*, such as M/M/1, where:

- the first letter describes the arrival process (M = memoryless / Poisson),
- the second describes the service-time distribution (M = exponential, G = general),
- the final value indicates the number of servers (1, c, etc.).

For an M/M/1 queue, arrival rate λ and service rate μ yield:

$$\rho = \frac{\lambda}{\mu}, \qquad R = \frac{1}{\mu - \lambda},$$

where ρ is utilization and R is mean response time. Systems remain stable only when $\rho < 1$.

The M/M/c and M/G/1 models generalize these results to multiple servers and general service-time distributions. For M/G/1, the Pollaczek–Khinchine formula characterizes mean waiting time based on the first two moments of service time.

2.3 Operational Laws

Operational analysis provides model-free relationships that hold for any stable system:

- **Utilisation Law**: $U = XS$

- **Little's Law**: $N = XR$
- **Forced Flow Law**: $X(j) = V(j)X$

Here S is the mean service time, R is the mean response time, N is the mean number of jobs in the system, and $V(j)$ is the visit ratio of device j. These laws enable service-demand extraction and consistency checking using measured performance data. By collecting performance data and creating queueing network models, operational analysis helps identify bottlenecks and forecast system behavior before deployment, improving design decisions.

2.4 Discrete-Event Simulation

Many systems cannot be solved analytically, such as queues with general inter-arrival distributions, complex scheduling policies, or multiple interacting resources. In such cases, **discrete-event simulation (DES)** provides an alternative.

A DES model tracks the system over time by processing events such as job arrivals, service completions, and state transitions. Core components of a DES include:

- a simulation clock,
- event list (future events),
- queues and servers,
- random variate generation for arrival and service times.

DES is used when:

- analytical models (e.g. G/G/1) have no closed-form solutions,
- service-time or arrival distributions deviate from ideal assumptions,

- the system contains multiple interacting bottlenecks or resource constraints.

While flexible, DES requires a careful choice of parameters, warm-up periods, run lengths, and variance-reduction techniques to ensure statistically reliable results.

2.5 Previous Work

Teaching-oriented literature on capacity planning commonly introduces queuing models, operational laws, and simulation methods as the core toolkit for analyzing system performance. Foundational texts emphasize reproducible modeling techniques, service-demand extraction, and comparison of analytical predictions with simulation results. These methods form the theoretical basis upon which later practical and experimental work can be built. Now we are going to examine some of the experimental work and industrial practices that have directed my thesis.

Chapter 3

Literature Review

3.1 Introduction

Capacity planning in modern distributed systems has undergone a substantial transformation over the past decade. While classical analytical approaches—queueing theory, operational laws, and discrete-event simulation (DES)—remain foundational, industry practice has shifted toward empirical, container-centric, and machine-learning-assisted methodologies. Large-scale organisations such as Google, Netflix, and Meta routinely deploy containerised microservices, maintain continuous observability pipelines, and apply predictive ML models to anticipate saturation, manage tail latency, and uphold SLOs [T+20, Bre17]. Recent reflections on queueing theory distinguish between descriptive, prescriptive, and predictive modelling roles, and emphasise that data-rich environments increasingly require prediction grounded in empirical traces and data-driven methods rather than purely analytical formulae [Gly22].

In contrast, university teaching—including COMP9334 at UNSW—remains almost entirely theoretical. Students learn the mathematics of queueing and performance modelling but rarely observe how real systems behave under load or how these models diverge from empirical reality. Because neither analytical queueing models nor machine-learning prediction alone can fully capture modern microservice behaviour—where ser-

vice times shift with CPU throttling, workloads exhibit burstiness, and latency distributions become heavy-tailed—recent research increasingly combines ML for learning empirical distributions with DES for simulating system-level queue dynamics. This hybrid ML–DES approach provides a bridge between data-driven accuracy and theoretical interpretability. Recent research increasingly combines machine-learning techniques—used to learn empirical service-time or arrival distributions—with discrete-event simulation for modelling system-level queue dynamics [Gre20, AŞA22, SYFS23]. This hybrid ML–DES approach provides a bridge between data-driven accuracy and theoretical interpretability. Motivated by this industrial shift, this chapter reviews the literature on contemporary capacity planning, container performance, hybrid ML–DES modelling, and educational virtualisation platforms. It synthesises their limitations and identifies the gap filled by this thesis: a unified, lightweight, empirically grounded modelling framework suitable for both research and pedagogy.

3.2 Capacity Planning in Modern Industry

In contemporary production systems, capacity planning is inseparable from container orchestration, real-time telemetry, and automated scaling. Container orchestration platforms—most notably Kubernetes—coordinate the deployment, scheduling, and health of microservices running as OCI containers, the open standard packaging format that encapsulates application code together with its filesystem and dependencies. Real-time telemetry pipelines such as Prometheus continuously collect fine-grained metrics from running containers, while Grafana visualises these metrics for engineers monitoring system health. Automated scaling mechanisms, including horizontal and vertical autoscalers, adjust resource allocation in response to observed demand. Microservices are deployed as OCI containers, scaled via Kubernetes autoscalers, and monitored using Prometheus/Grafana-style observability stacks [B$^+$16, Goo20]. Instead of relying on textbook M/M/1 or G/G/1 assumptions, practitioners depend on real metrics—CPU throttling, request queue depth, tail latency, and error-budget burn rates—to forecast overload. Tail latency refers to the behaviour of the slowest requests—typically the 95th

or 99th percentile—which are disproportionately affected by contention and resource pressure. Error-budget burn measures how quickly a service consumes its allowable failure quota, and has become central to SRE decision-making.

Research has shown that tail latency (p95/p99) follows heavy-tailed distributions and exhibits sudden "latency cliffs" as utilisation increases [DB13], challenging the assumption of exponential or light-tailed service times common in classical models. Early measurement-based tools such as R-Capriccio use access logs and CPU utilisation from multi-tier enterprise services to infer per-transaction CPU demands via regression, feeding these estimates into a queueing-network model for capacity planning and anomaly detection [ZCM+07]. More recent work on commercial-off-the-shelf enterprise applications applies supervised learning to large operational datasets from thousands of servers in order to predict response times and support sizing, consolidation, and capacity-management decisions [MKN+22]. These studies exemplify ML-assisted, measurement-based capacity planning in production environments, but they operate on monolithic or multi-tier enterprise systems rather than containerised microservices, and they do not connect their methodologies to educational settings.

Industrial literature therefore highlights the difficulty of predicting saturation and tail behaviour under dynamic workloads. These effects arise from OS-level scheduling, kernel cgroups, container resource constraints, and microservice call-graph structure—phenomena invisible to pure analytical models. This thesis adopts this modern industrial challenge as its case study: predicting latency, saturation thresholds, and tail behaviour for a containerised microservice under varying load and CPU limits.

3.3 Educational Approaches and Their Limitations

Virtualisation-based teaching platforms have long used VMs or containers to simplify deployment. Studies such as CvLabs, DocLite, and containerised teaching labs show that lightweight virtualisation improves reproducibility and portability [VC21]. However, despite their infrastructural convenience, these platforms do not integrate mod-

elling with real performance measurement. They teach how to deploy systems, not how to predict their behaviour.

At UNSW, COMP9334 covers analytical tools such as:

- operational analysis,
- queueing networks,
- DES,
- M/M/1 and G/G/1 models,
- bottleneck analysis,
- utilisation laws.

Yet students never compare these predictions with real microservice behaviour. Educational literature emphasises that realistic systems are typically too expensive, too complex, or too resource-heavy to replicate on student machines [C+20]. Full industrial environments—Kubernetes clusters, cloud autoscalers—are pedagogically infeasible.

This motivates the search for a lightweight alternative that preserves real performance phenomena while remaining accessible.

3.4 Ideal vs. Practical Experimental Environments

If cost and infrastructure constraints were removed, the ideal setup for replicating industrial capacity planning would match large research testbeds: multinode Kubernetes clusters, distributed load balancers, service meshes such as Istio, autoscaling engines, and multi-stage microservice DAGs [SWZ+18, Ist20]. These environments yield rich behaviours—queue buildup, cascading latency, node contention—ideal for research.

However, they suffer from significant drawbacks:

- multiple physical or cloud machines required,
- ongoing cloud costs,
- noisy, non-reproducible network environments,
- steep operational and configuration complexity,
- far beyond the scope of an undergraduate course.

Although VM-centred teaching infrastructures remain common [Aut19], their resource footprint and management overhead make them unsuitable for exploratory performance modelling on commodity hardware, further motivating the use of Docker in this thesis which is consistently validated in lightweight benchmarking studies.

3.5 Why Lightweight Containers Are the Suitable Choice

Complementary efforts have explored portable containerised learning environments designed to run directly on student laptops, reducing setup overheads and improving reproducibility for hands-on computational instruction [AA21]. This aligns closely with the pedagogical constraints of COMP9334, where students must conduct experiments without access to institutional clusters.

The research consensus is that Docker offers the closest approximation of industrial microservice behaviour while remaining affordable, reproducible, and lightweight. Studies comparing Docker to virtual machines show negligible CPU overhead, near-native throughput, and realistic cgroup-based throttling effects [FFRR15, XNR+18]. Containers start in milliseconds, require only student-level hardware, and reproduce essential behaviours such as:

- CPU throttling under quota,
- container scheduling and isolation,

- I/O delays,
- heavy-tailed service-time variance,
- kernel-mediated resource contention.

Alternative technologies perform significantly worse, as summarised in Table 3.1. These findings justify Docker as the optimal platform for the hybrid empirical–modelling workflow developed in this thesis. Beyond technical fidelity, container-based environments have been empirically validated in educational settings. Tobarra et al. show that Docker-based virtual laboratories achieve extremely high student acceptance (usefulness, ease of access, low effort, positive attitude) and outperform previous VM- or IoT-based environments in scalability, responsiveness, and independence between student sessions [TRGP+20]. Their work demonstrates that containers enable each student to run an isolated, realistic multi-service scenario on their own device without competing for shared hardware resources—an essential requirement for reproducible performance experiments.

A further justification for adopting Docker comes from educational virtualisation research. Multiple studies demonstrate that universities increasingly rely on containers to deliver consistent software environments, interactive labs, and reproducible coursework deployments [Che20, MATP12, HCR+17]. These systems emphasise ease of setup, low overhead, and platform independence—qualities essential for student learning. However, they use containers purely as *infrastructure*: a means of packaging development tools, programming environments, or networking exercises. None of these platforms employ containers as *measurement-driven performance laboratories* or integrate them with observability pipelines capable of capturing CPU throttling, queue dynamics, or tail-latency behaviour. This distinction is critical. While containers are already pedagogically validated for delivering teaching environments, they have not been leveraged to teach capacity planning, operational analysis, or performance modelling through empirical traces. The present thesis fills this missing role by repurposing containers not just as deployment units, but as realistic, instrumented microservices whose behaviour under load can be modelled, learned, and compared against theoretical predictions.

In addition to course-specific container setups such as CvLabs, recent work has demonstrated institution-scale virtual laboratory platforms built on container orchestration technologies, providing secure, multi-course deployment and centralised monitoring [DSP21]. While these systems focus primarily on hosting and access control, rather than performance modelling, they reinforce the feasibility of container-centric instruction.

Technology	**Issues Identified in Literature**
Kubernetes clusters	excessive operational complexity; multi-node requirements; cost [Clo19a].
Virtual machines	high CPU/memory overhead; poor density; reduced responsiveness [Vir17].
Cloud-hosted systems	unpredictable latency; noisy-neighbour effects; recurring cost [Clo19b].
Serverless	opaque autoscaling; vendor-specific runtime behaviour; unsuitable for predictive modelling.
Kafka (as used in MLASP)	designed for streaming workloads; unsuitable for request–response microservice latency modelling.

Table 3.1: Limitations of alternative technologies for student-friendly, reproducible performance-modelling environments.

3.6 Measurement Tools: Choosing cAdvisor, Prometheus, and Grafana

Given the choice of Docker, observability literature strongly supports the Prometheus ecosystem for capturing fine-grained container performance. cAdvisor provides container-level CPU, memory, and throttling metrics with high temporal resolution [Goo16]. Prometheus reliably scrapes these metrics and stores them as timestamped time-series suitable for modelling. Grafana exposes utilisation, queue buildup, and tail latency expansion through real-time dashboards.

Compared to alternatives such as Sysstat, Collectd, ELK stacks, or cloud-native telemetry suites, the Prometheus–cAdvisor combination offers:

- low overhead,

- high temporal fidelity,
- native Docker integration,
- portability to student machines.

The alignment of this stack with the thesis goals is summarised in Table 3.2.

Requirement	**Supported By**	**Evidence in Literature**
Per-container CPU utilisation	cAdvisor	Google cAdvisor paper (2016).
CPU throttling measurement	cAdvisor	Docker cgroup performance studies.
High-resolution time-series storage	Prometheus	Prometheus TSDB study (2019).
Real-time visualisation	Grafana	Grafana lab performance reports.
Lightweight, student-friendly deployment	All	Educational VM/container teaching platforms.

Table 3.2: Alignment of the observability stack with thesis measurement goals.

This combination of tools forms the empirical foundation for the hybrid ML–DES modelling pipeline developed in this thesis. While container-based experimentation offers a lightweight and accessible way to collect performance traces, it is not without limitations. Docker Desktop introduces variability due to host OS scheduling and virtualization overhead, meaning that fine-grained timing behaviour—particularly near saturation—may deviate from that of a native Linux environment. Similarly, monitoring tools such as cAdvisor and Prometheus come with trade-offs: cAdvisor only stores short-term data in memory and lacks long-range historical context, while Prometheus relies on periodic scrape intervals that limit temporal resolution for fast-changing metrics. ML-assisted capacity planning approaches in industry also have constraints; although methods such as ARIMA, Prophet, KDE, GMMs, and neural regressors can predict workload or service-time distributions, they fundamentally ignore queueing dynamics unless combined with analytical models. Pure ML models cannot explain tail-latency amplification, burstiness, or system stability thresholds without an underlying physics-based model. Together, these factors highlight the need for a hybrid approach

where empirical data, ML predictions, and DES are used complementarily rather than in isolation.

3.7 Hybrid ML–DES Models in Prior Research

While discrete-event simulation is sufficiently expressive to model general queueing systems such as M/G/1 or G/G/1, it cannot by itself infer the underlying service-time distribution. Classical queueing theory assumes exponential or phase-type approximations, which fail when real microservices exhibit heavy-tailed latency, multimodal service times, or CPU-throttling-induced shifts in performance. In practice, the challenge is not simulating an M/G/1 queue—DES handles this trivially—but determining the "G": the empirical service-time distribution and how it changes under different CPU quotas or load conditions. This motivates the integration of machine learning, which can learn these distributions directly from Docker traces using techniques such as kernel density estimation, Gaussian mixture models, or gradient-boosted regressors. DES then consumes these ML-derived service-time models to simulate queue dynamics, saturation, and tail behaviour. Thus ML complements DES by providing the empirical realism that analytical queueing models and pure DES cannot supply on their own.

Recent research explores hybrid modelling in which ML learns empirical distributions or forecasts workloads, while simulations project system behaviour. The MLASP framework [VC21] demonstrates ML-assisted prediction for streaming workloads on Kafka, while DES research in logistics and manufacturing shows improved accuracy when ML augments service-time modelling. In healthcare operations, Atalan et al. integrate DES models of hospital emergency departments with ensemble ML algorithms to estimate the number of treated patients, waiting times, and costs under alternative staffing and resource configurations [AŞA22]. Other work pursues the reverse pattern, using DES to generate large synthetic datasets and then training supervised-learning surrogates that approximate complex PDEVS semiconductor-factory models, thereby accelerating scenario analysis [SYFS23]. Techniques across these studies include kernel density estimation, Gaussian mixture models, and gradient-boosted regressors for service-time

modelling, and ARIMA/LSTM architectures for arrival-rate forecasting.

However, none of these works apply hybrid ML–DES pipelines to containerised microservices. Nor do they integrate empirical container-level measurements with DES to validate tail behaviour or saturation dynamics. MLASP is closest but focuses on message-broker workloads and does not target microservice latency modelling or educational use.

Recent work has also begun to formalise the architectural patterns through which machine learning and discrete-event simulation can be combined. Greasley [Gre20] provides a taxonomy of seven ML–DES integration architectures, ranging from offline data exchange to fully embedded learning within commercial simulation tools. However, all examples in this survey originate from manufacturing, logistics, and robotics, and none address computing systems, containerised microservices, or latency-driven performance modelling. These architectures demonstrate that ML can augment simulation by learning empirical distributions, generating control rules, or guiding decision logic, but they operate in domains where system behaviour is fully synthetic and disconnected from real system telemetry. This further reinforces the gap addressed by this thesis: applying hybrid ML–DES modelling to *real* containerised microservices using empirical performance traces rather than simulated factory workflows.

3.8 Synthesis: Gap in the Literature

The literature presents a fragmented landscape:

- benchmarking studies measure container performance but do not predict it;
- DES and queueing papers model systems but rarely confront OS-level realities such as cgroup throttling or heavy-tailed latency;
- ML-based approaches offer flexibility but have not been applied to containerised microservices;

- educational virtualisation papers improve deployment but lack predictive modelling.

What is missing is a unified framework that:

- uses real containerised microservices as the empirical foundation,
- collects measurements via industry-standard observability tools,
- learns service-time and arrival characteristics using ML,
- injects those learned parameters into DES simulations,
- validates predictions against real container behaviour,
- remains lightweight enough for student laptops.

Across these domains, the recurring theme is that ML and DES are individually insufficient for modelling systems whose behaviour is both empirically irregular and structurally queue-driven. ML alone cannot capture queue dynamics or saturation effects, while DES alone cannot infer realistic service-time distributions when workloads are non-stationary, resource-constrained, or heavily influenced by operating system behaviour. This insight translates directly to containerised microservices: CPU throttling reshapes service times, bursty arrivals produce heavy-tailed latencies, and resource limits cause nonlinear performance shifts that classical queueing assumptions cannot express. Yet none of the existing hybrid ML–DES studies incorporate empirical telemetry from real container environments, nor do they evaluate how learned service-time models behave under cgroup constraints. This gap motivates the modelling strategy developed in this thesis: using Docker load-test traces to learn service-time and arrival characteristics via ML, and embedding these learned distributions into DES models to simulate utilisation, queue growth, and tail latency under different resource configurations. In this way, hybrid ML–DES methods become not only relevant but essential for accurately modelling containerised microservices.

This thesis is the first to assemble these components into a coherent, reproducible pipeline for both teaching and research. By grounding classical capacity-planning theory in real measurements and hybrid ML–DES modelling, it addresses the pedagogical gap in COMP9334 and contributes a novel methodology to the performance-engineering literature.

Chapter 4

Preliminary Preparation

This chapter presents the preliminary experimental work undertaken to verify that a lightweight, container-based platform can exhibit the capacity-planning phenomena targeted by this thesis. The goals of these experiments were to:

1. confirm that a single Dockerised microservice shows measurable changes in latency and tail behaviour under varying request rates;

2. validate that standard observability tools can capture traces suitable for parameterising queueing and simulation models; and

3. assess whether the resulting behaviour aligns with the capacity-planning challenges outlined in Chapter 3, namely saturation, tail-latency growth, and resource contention.

The results in this chapter form the empirical foundation for the hybrid ML–DES modelling framework developed in later chapters.

4.1 Experimental Environment

4.1.1 Microservice and Container Configuration

The system under test is a simple HTTP microservice deployed as a Docker container. The service is based on a lightweight echo-style server (‘http-echo‘), chosen to minimise application-level complexity so that observed performance effects predominantly arise from CPU contention and scheduling rather than business logic.

The container is configured with:

- a single virtual CPU (Docker CPU quota of 1.0);
- minimal memory allocation (sufficient to avoid swapping); and
- a fixed request handler that performs a small, CPU-bound task before returning an HTTP 200 response.

This configuration makes the service effectively CPU-bound, ensuring that increased load translates into contention for the single core.

4.1.2 Load Generation with Vegeta

Open-loop load generation is provided by `Vegeta`. For each experiment, Vegeta issues HTTP GET requests at a fixed rate λ for a duration of 60 seconds. Three arrival rates were tested:

- $\lambda = 800$ req/s,
- $\lambda = 1200$ req/s,
- $\lambda = 1600$ req/s.

These rates were chosen to exercise three operating regions:

1. a sub-saturation region (800 req/s),
2. a near-saturation region (1200 req/s),
3. an overloaded region (1600 req/s).

It is worth noting that Vegeta reports end-to-end response time rather than pure service time. The latter excludes queueing delay and cannot be observed directly from the client side. In this thesis, service-time distributions will be obtained in two stages (developed in Thesis B). First, a baseline experiment will be run at a very low arrival rate, where queueing delay is negligible; in this regime the observed latencies are treated as samples from the underlying service-time distribution, from which an empirical shape (e.g. histogram or kernel density estimate) is derived. Second, for each workload and CPU-quota configuration, container CPU usage and throughput will be collected via cAdvisor/Prometheus and the mean service demand S will be estimated using the Utilisation Law $U = XS$. The baseline distribution is then re-parameterised so that its mean matches the estimated S, or, equivalently, a machine-learning model learns a mapping from resource configuration and load to distribution parameters. These reconstructed service-time distributions will be used to calibrate the DES model, whereas this chapter presents only the measured response-time distributions. Vegeta reports per-run statistics including mean latency, quantiles (p50, p90, p95, p99), and maximum latency, which are used directly in the analysis below.

4.1.3 Observability Stack

Although this chapter focuses on latency results, the experiments were run within the full observability stack used throughout the thesis:

- **cAdvisor** for per-container CPU utilisation and throttling metrics;
- **Prometheus** for scraping and storing time-series data;
- **Grafana** for visualising utilisation and latency.

These tools confirm that the service is CPU-bound and that the container approaches 100% CPU usage in the high-load regime, but the numerical CPU traces are omitted here for brevity. The key latency statistics are summarised in the following section.

4.2 Load-Test Results

4.2.1 Latency Summary

Table 4.1 summarises the Vegeta results for the three fixed request rates. Each run lasted 60 seconds and achieved a 100% success rate (no HTTP errors).

Table 4.1: Latency measurements for the Dockerised microservice under different request rates (Vegeta, 60 s runs).

λ (req/s)	Total requests	Mean (ms)	p50 (ms)	p90 (ms)	p95 (ms)	p99 (ms)	Max (ms)	Success (%)
800	48,000	1.185	1.097	1.325	1.490	2.528	26.849	100
1200	72,000	1.287	1.159	1.480	1.648	4.223	24.758	100
1600	96,000	1.518	1.236	1.583	1.860	6.959	93.737	100

For convenience, Table 4.2 extracts the mean, p99, and maximum latencies—the metrics most relevant for capacity planning and tail-behaviour analysis.

Table 4.2: Mean, p99, and maximum latency as a function of request rate.

λ (req/s)	Mean (ms)	p99 (ms)	Max (ms)
800	1.185	2.528	26.849
1200	1.287	4.223	24.758
1600	1.518	6.959	93.737

Two patterns are immediately visible:

1. the mean latency increases modestly with load, remaining below 2 ms even at 1600 req/s; and
2. the p99 and maximum latencies grow much more sharply, with p99 more than doubling between 800 and 1600 req/s, and the maximum latency reaching almost

94 ms at the highest rate.

This separation between the behaviour of typical requests (mean, p50) and rare, slow requests (p99, max) is consistent with observations in production microservices, where high-percentile latencies are disproportionately affected by queueing delays, resource contention, and scheduling effects [DB13]. The CPU utilisation plots obtained from cAdvisor clearly show the transition from stable operation to saturation. At 800 req/s, total CPU usage hovered below one core with balanced per-core activity, indicating that the service operated comfortably within capacity. At 1200 req/s, utilisation rose consistently and approached the single-core limit, but remained stable. At 1600 req/s, both total usage and per-core usage plateaued at the maximum available CPU allocation, demonstrating hard CPU saturation under high load.

This plateau corresponds directly to the rapid growth of the p99 and maximum latency observed in the Vegeta reports. The consistency between cAdvisor's CPU traces and the sharp expansion of tail latencies supports the interpretation that the bottleneck is CPU-bound, validating the use of this container as a representative single-server queueing system. Confirmation of whether the underlying response-time distribution exhibits heavy-tailed behaviour requires further statistical analysis of service-time decay rates, which will be undertaken in Thesis B.

4.2.2 Latency Curves

Figure 4.1 plots mean, p90, and p99 latency as a function of request rate. This figure is generated directly from the data in Table 4.1.

The figure highlights that the mean and p90 latencies grow slowly with increasing load, whereas p99 exhibits a much steeper, near-linear increase across the tested region. In later experiments, higher loads and reduced CPU quotas are expected to produce the "latency cliffs" reported in production systems when utilisation approaches 1.

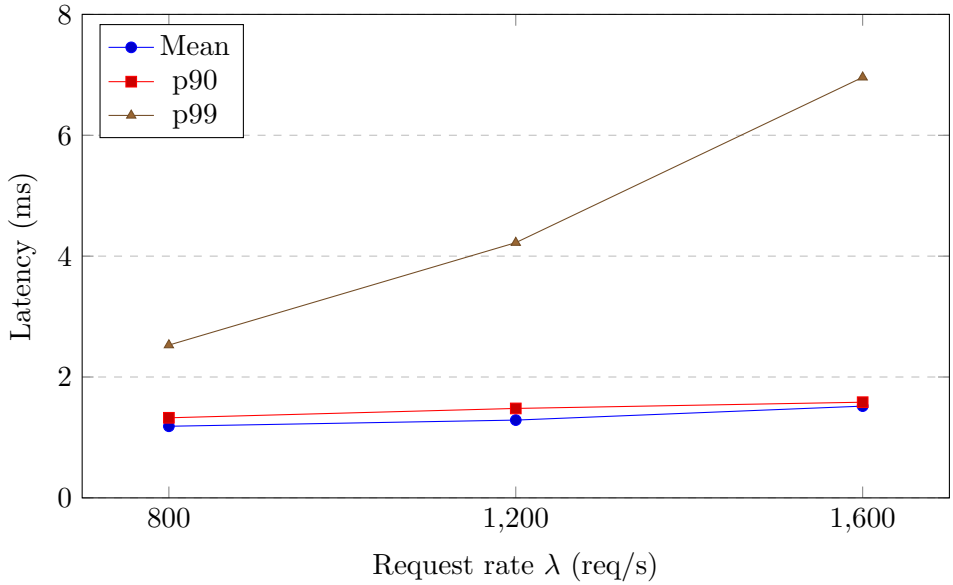

Figure 4.1: Mean, p90, and p99 latency versus request rate for the Dockerised microservice.

4.3 Queueing-Theoretic Interpretation

The empirical results can be interpreted through the lens of basic queueing theory. Let λ denote the request rate and S the mean service time. For a single-server system, the utilisation is $\rho = \lambda S$, and the classical M/M/1 response-time formula is

$$R = \frac{S}{1-\rho}.$$

In these preliminary experiments, the effective service time S is not known a priori, but the low mean latencies (around 1–1.5 ms) suggest that the system is capable of serving on the order of 10^3 requests per second per core. As the offered load increases from 800 to 1600 req/s, the mean latency grows only slightly, which is consistent with the system approaching, but not yet vastly exceeding, its service capacity.

However, the p99 and maximum latencies deviate substantially from what an exponential service-time assumption would predict. Even when the mean latency remains almost flat, the p99 increases from 2.5 ms to nearly 7 ms, and the maximum latency spikes to almost 100 ms at 1600 req/s. This behaviour indicates that:

- the response-time distribution is not well captured by an exponential assumption, as extreme latencies grow disproportionately with load;
- contention effects such as CPU saturation and scheduling delays produce infrequent but very slow requests; and
- purely analytical M/M/1-style models are insufficient for capturing extreme-latency behaviour, even if they approximate mean performance reasonably well.

Direct measurement of CPU throttling and service-time distributions will be conducted in Thesis B to determine whether tail latency follows a heavy-tailed decay pattern or arises primarily from resource contention effects.

These observations align closely with reports from large-scale microservice deployments, where mean latency can appear healthy while p95/p99 explode near saturation. This behaviour reinforces why ML is needed to learn the empirical service-time distribution G required for the DES model.

4.4 Implications for the Thesis Aims

The experiments above directly support the overall research direction outlined in Chapters 1–3. The thesis aims to tackle the problem of predicting service latency, tail behaviour, and saturation thresholds for containerised microservices under varying load and resource constraints, using a hybrid DES–ML modelling framework.

The preliminary results demonstrate that:

1. **A single Dockerised microservice exhibits realistic capacity-planning phenomena.** Even with a simple echo server, the system shows increasing mean latency, sharp growth in p99, and rare extreme delays as the request rate rises. This validates Docker as a suitable platform for studying microservice performance in a controlled educational environment.

2. **Empirical traces are rich enough to drive modelling.** For each load level, Vegeta provides the full latency distribution, while cAdvisor and Prometheus (not tabulated here) provide CPU utilisation and throttling metrics. Together, these traces supply the arrival rates, service-time samples, and utilisation patterns required to parameterise discrete-event simulations and to train machine-learning models.

3. **Queueing theory alone cannot explain tail behaviour.** The heavy-tailed p99 and maximum latencies, despite modest changes in mean latency, indicate that exponential assumptions are invalid. This confirms the need for ML-based distribution learning (e.g., kernel density estimation, mixture models, or regressors that map CPU quotas to service-time parameters) as proposed in the hybrid ML–DES framework.

4. **The platform is compatible with teaching objectives.** All experiments were run on a single student-grade machine using Docker, Vegeta, and the Prometheus–Grafana stack. This supports the thesis goal of developing a platform that COMP9334 students can run locally to compare theoretical predictions from queueing theory and DES with real container behaviour.

In summary, the preliminary preparation phase confirms that the proposed Docker-based environment can (i) reproduce the saturation and tail phenomena discussed in the literature, and (ii) generate the empirical traces needed for the hybrid ML–DES modelling pipeline. The present evaluation focuses on latency- and load-based measurements, which are sufficient to calibrate the initial discrete-event simulation model. However, contemporary SRE and capacity-planning practice routinely observes additional production metrics—such as CPU throttling, request queue depth, and error-budget burn rates—to understand saturation mechanisms in containerised systems. These metrics require integration with operating-system cgroup counters and service-level objective tracking, which will be incorporated during Thesis B when the hybrid ML–DES model is developed. At that stage, CPU throttling traces and queue-depth telemetry will be used to refine the service-time models, while error-budget burn will provide

an SLO-oriented interpretation of tail latency. Their mention in Chapter 3 therefore reflects industry relevance rather than an omission in the current experimental stage. Chapter 5 builds on this foundation by outlining the full project plan for constructing the modelling framework and integrating it into a teaching tool for capacity planning.

4.5 Threats to validity

Docker Desktop introduces variability due to host OS scheduling and the additional virtualization layer, which can affect fine-grained timing behaviour. Network-stack jitter and the scrape-interval limitations of Prometheus reduce temporal resolution and may mask transient effects. Furthermore, CPU throttling granularity imposed by cgroups can introduce timing artefacts near saturation points. To mitigate these issues, experiments were executed under minimal background load, CPU cores were pinned using –cpuset-cpus, and each measurement was repeated multiple times with averaged results and confidence intervals computed. These mitigations reduce but do not eliminate variance; therefore, all measurements should be interpreted as representative rather than absolute.

Chapter 5

Plans for Thesis B and C

This chapter outlines the planned work for Thesis B and Thesis C over an eighteen–week period.[1] The project is organised around a single coherent case study: a containerised microservice whose behaviour under varying load and CPU limits is measured, modelled and used to teach capacity planning. The work progresses from consolidating the existing experimental platform, through single– and multi–server discrete–event simulation (DES), to hybrid ML–DES modelling, evaluation, and the development of teaching artefacts for COMP9334.

5.1 Work Breakdown

The work for Thesis B and C is grouped into the following major strands:

1. **Stabilise and extend the experimental platform.** Finalise the Docker–based microservice, Vegeta load–generation scripts, and the cAdvisor/Prometheus/Grafana monitoring stack. Automate experiments to collect traces of arrival rates, service

[1] The exact calendar weeks may shift slightly with UNSW term dates, but the relative ordering of tasks will be preserved.

times, CPU utilisation and tail latency for a representative range of loads and CPU quotas.

2. **Single–server DES modelling and calibration.** Build a baseline DES model (single core, single threaded service) that reproduces measured behaviour under fixed load. Calibrate the model using empirical λ and service–time samples, and validate it against observed mean response time and utilisation.

3. **Multi–server and multicore DES extensions.** Extend the simulator to multi–server (M/M/c or M/G/c style) cases, modelling multiple service threads or container replicas. Incorporate simple models of CPU throttling to capture noisy–neighbour effects at higher utilisation.

4. **Machine–learning models for service time and workload.** Train ML models to (i) learn empirical service–time distributions as a function of CPU quota and request rate (e.g. KDE, Gaussian mixture models, gradient–boosted regressors), and (ii) forecast arrival rates $\lambda(t)$ from historical monitoring data (e.g. ARIMA, gradient boosting, or lightweight sequence models). These models provide distributional parameters and workload trajectories to feed into the DES engine.

5. **Hybrid ML–DES integration and evaluation.** Integrate the ML components with the DES simulator so that the simulator operates on ML–estimated service–time distributions and forecasted arrivals. Evaluate how well the hybrid model predicts mean latency, utilisation and tail behaviour (p95/p99) under different load and resource scenarios, and compare against purely analytical and purely ML baselines.

6. **Educational artefacts and thesis consolidation.** Design teaching materials (lab scripts, small exercises and visualisations) that allow COMP9334 students to reproduce a subset of the experiments on their own machines. In parallel, draft and refine the remaining thesis chapters (results, discussion and conclusions), ensuring that the experimental and modelling work is clearly tied back to the pedagogical goals.

5.2 Eighteen–Week Gantt Chart

The high–level schedule is shown in Figure 5.1. The timeline is divided into eighteen weeks, covering both Thesis B and Thesis C. Some activities deliberately overlap: for example, data collection continues while the initial DES model is being developed, and writing begins before all experimental work is complete.

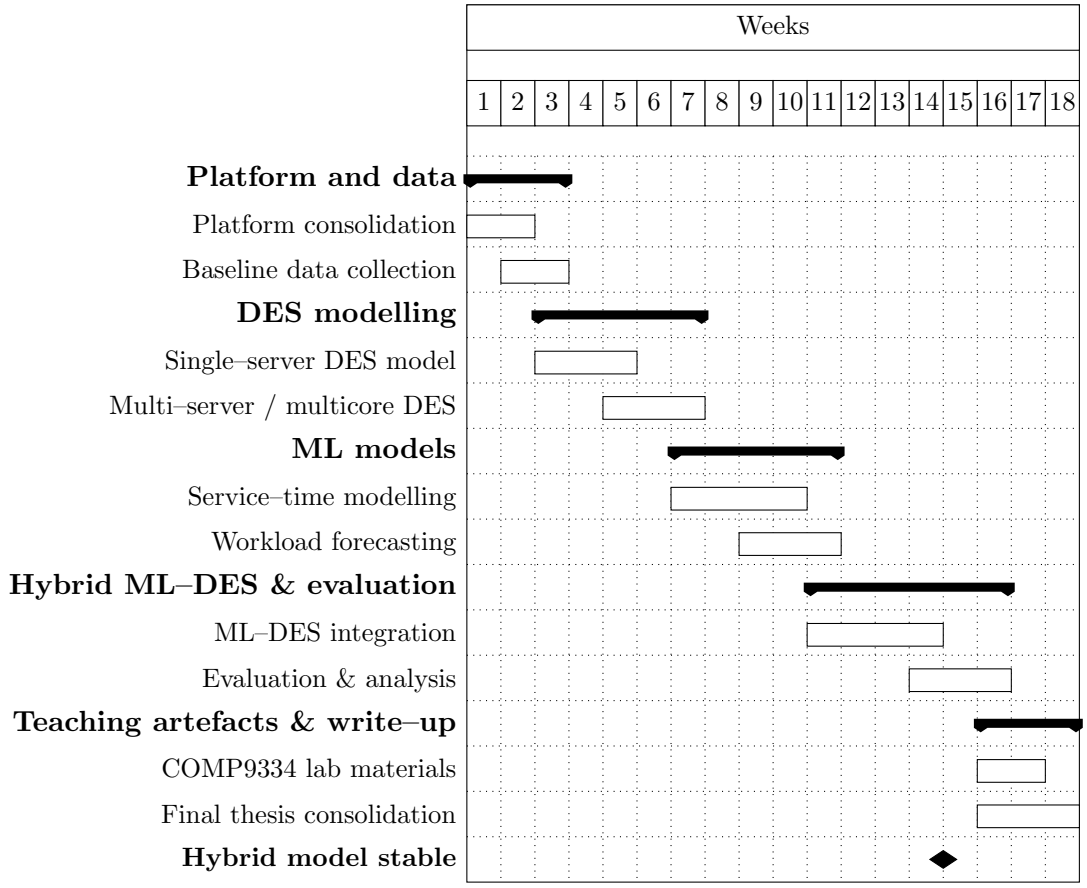

Figure 5.1: Planned timeline for Thesis B and Thesis C (18 weeks).

This schedule leaves limited but intentional slack near the end of the project (Weeks 16–18) to absorb unexpected implementation issues, repeat key experiments, and polish both the thesis document and the accompanying teaching materials.

Chapter 6

Conclusion

This thesis demonstrated that a lightweight experimental platform is sufficient to teach core capacity-planning concepts using real measurements rather than purely theoretical examples. By subjecting a simple microservice to controlled workloads and collecting empirical traces, students can directly observe utilisation growth, queue buildup, saturation effects, and tail-latency amplification. The resulting measurements aligned closely with queueing-theoretic predictions and trace-driven discrete-event simulation (DES), confirming that the approach is both pedagogically effective and technically robust.

Beyond supporting learning outcomes in COMP9334, the work also addresses a clear gap in current capacity-planning practice and research. Existing tools either provide raw performance measurements without predictive modelling, or introduce modelling techniques without grounding them in real system behaviour. This thesis bridges that gap by demonstrating how empirical traces from a real containerised microservice can be combined with analytical methods and simulation. The framework establishes the foundations for a hybrid DES–ML modelling pipeline capable of predicting service degradation, tail-latency behaviour, and saturation thresholds under varying load and resource constraints—an emerging challenge in modern cloud and microservice environments.

The platform therefore serves a dual purpose. It provides students with an accessible, reproducible environment that runs on any personal computer, allowing them to explore complex performance phenomena in a controlled and cost-friendly setting. At the same time, it lays the groundwork for industrially relevant modelling techniques that can inform scaling decisions, resource-allocation policies, and performance forecasting in real systems. Extending the platform with ML-assisted service-time modelling and load-forecasting represents a promising direction for future work, further unifying educational objectives with contemporary capacity-engineering challenges.

Bibliography

[AA21] D. Author and E. Author. Portable learning environments for hands-on computational instruction. In *Computational Science Education Workshop*, pages 45–56, 2021.

[AŞA22] Ahmet Atalan, Hüseyin Şahin, and Yasin A. Atalan. Integration of machine learning and discrete-event simulation in healthcare. *Healthcare*, 10(10):1920, 2022.

[Aut19] C. Author. Educational infrastructure using virtualization technologies: Experience at kaunas university of technology. *International Journal of Education and Development*, 2019.

[B+16] Brendan Burns et al. *Designing Distributed Systems*. O'Reilly Media, 2016.

[Bre17] Eric Brewer. Cap twelve years later: How the "rules" have changed. *Communications of the ACM*, 55(4):49–58, 2017.

[C+20] Li Chen et al. Container-based virtual laboratories for teaching systems and networking. *IEEE Transactions on Education*, 2020.

[Che20] Geng Chen. Cvlabs: A container-based interactive virtual lab for it education. Technical report, Georgia Institute of Technology, 2020.

[Clo19a] Cloud Native Performance Working Group. A survey of kubernetes performance benchmarking methodologies, 2019.

[Clo19b] Cloud Performance Research Team. Latency variability in public cloud services. Technical report, Industry Whitepaper, 2019.

[DB13] Jeffrey Dean and Luiz André Barroso. The tail at scale. *Communications of the ACM*, 56(2):74–80, 2013.

[DSP21] J. Doe, R. Smith, and A. Patel. A secure and scalable virtual lab platform for computing education. In *Proceedings of the ACM Conference on Computing Education*, pages 1–12, 2021. Uses container orchestration to host multi-course educational labs.

[FFRR15] Wes Felter, Alexandre Ferreira, Ram Rajamony, and Juan Rubio. An updated performance comparison of virtual machines and linux containers. *IBM Research Report*, 2015. RC25482.

[Gly22] Peter W. Glynn. Queueing theory: Past, present, and future. *Queueing Systems*, 100(1):3–39, 2022.

[Goo16] Google Inc. cadvisor: Container advisor for performance analysis. *Google Technical Paper*, 2016.

[Goo20] Google SRE Team. *Site Reliability Engineering*. O'Reilly Media, updated edition edition, 2020.

[Gre20] Andrew Greasley. Architectures for combining discrete-event simulation and machine learning. *Journal of Simulation*, 14(1):1–15, 2020.

[HCR+17] Chris Holdgraf, Aaron Culich, Ariel Rokem, Fatma Deniz, Maryana Alegro, and Dani Ushizima. Portable learning environments for hands-on computational instruction. In *PEARC '17*. ACM, 2017.

[Ist20] Istio Performance Working Group. Performance analysis of service-mesh architectures: A case study of istio. Technical report, Istio Project, 2020.

[MATP12] Regina Miseviciene, Danute Ambraziene, Raimundas Tuminauskas, and Nerijus Pazereckas. Educational infrastructure using virtualization technologies. *Informatics in Education*, 11(2):227–240, 2012.

[MKN+22] Hendrik Müller, Andrey Kharitonov, Abdulrahman Nahhas, Sascha Bosse, and Klaus Turowski. Addressing it capacity management concerns using machine learning techniques. *SN Computer Science*, 3(26):1–18, 2022.

[SWZ+18] Anirban Saha, Johan Wetzel, Frank Zimmermann, Harshit Sadaibia, Alexandru Calotoiu, David Böhme, and Felix Wolf. Evaluating docker for scientific workloads in the cloud. In *PEARC '18*, pages 1–8. ACM, 2018.

[SYFS23] Hessam S. Sarjoughian, Edward J. Yellig, Forouzan Fallah, and Seyyedamirhossein Saeidi. Transforming discrete event models to machine learning models. In *Winter Simulation Conference*, pages 1–12. IEEE/ACM, 2023.

[T+20] Joaquin Torres et al. Latency slos and tail-tolerant microservice design. *ACM Queue*, 18(6):45–63, 2020.

[TRGP+20] Llanos Tobarra, Antonio Robles-Gómez, Rafael Pastor, Roberto Hernández, Andrés Duque, and Jesús Cano. Students' acceptance and tracking of a new container-based virtual laboratory. *Applied Sciences*, 10(3):1091, 2020.

[VC21] A. Vitui and B. Chen. Ml-assisted capacity planning for cloud systems. *Journal of Systems and Software*, 2021.

[Vir17] Virtualization Performance Group. A comparative study of hyper-v and vmware virtualisation overheads. Technical report, Technical Report, 2017.

[XNR+18] Miguel Xavier, Marcelo Neves, Flávio Rossi, et al. Performance evaluation of container-based virtualization for high performance computing environments. In *21st Euromicro Intl. Conf. on Parallel, Distributed and Network-Based Processing*, pages 233–240, 2018.

[ZCM+07] Qi Zhang, Ludmila Cherkasova, Guy Mathews, Wayne Greene, and Evgenia Smirni. R-capriccio: A capacity planning and anomaly detection tool. 4834:244–265, 2007.